

# 22CS61- OPERATING SYSTEM CONCEPTS

## UNIT-1

# Chapter 1: Introduction

- What Operating Systems Do
- Operating-System Operations

## Chapter 2: Operating System Structures

- Operating System Structure

## Chapter 3. Process Management- Processes

- Basic concept
- Process scheduling
- Operations on processes
- Inter process Communication

## Chapter 4. Threads and Concurrency

- Overview
- Multithreading models

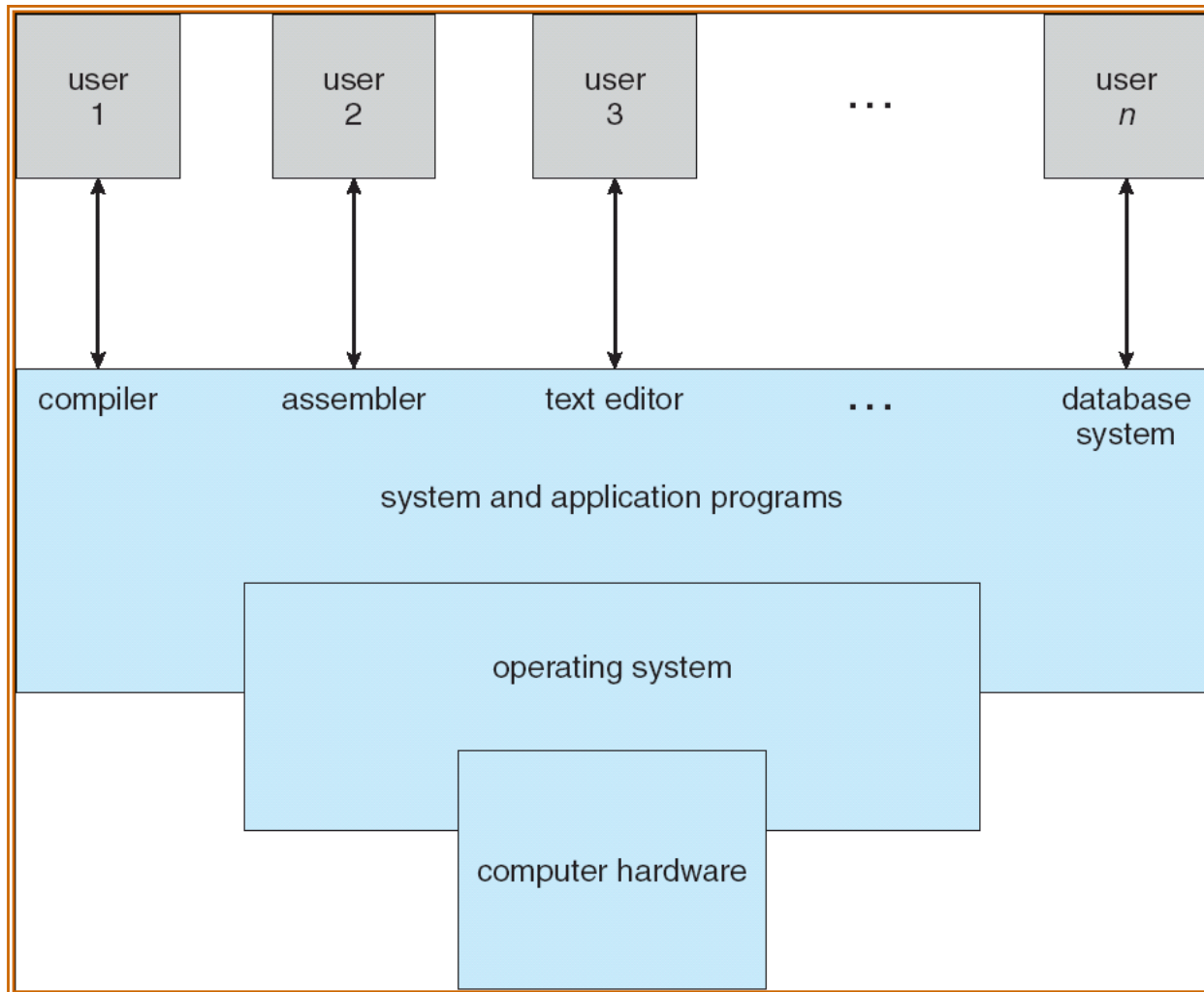
## COURSE LEARNING OUTCOMES

- **CO1:** Explain the fundamental concepts, structure, and operations of Operating Systems along with process management and multithreading approaches.

# What is an Operating System?

- A program that acts as an **intermediary** between a user of a computer and the computer hardware.
- Operating system goals:
  - Execute user programs and make solving user problems easier.
  - Make the computer system convenient to use.
- Use the computer hardware in an efficient manner.

# Four Components of a Computer System



# System view

- OS is a **resource allocator**
  - Manages all resources
  - Decides between conflicting requests for efficient and fair resource use
- OS is a **control program**
  - Controls execution of programs to prevent errors and improper use of the computer

# Operating System Definition

- No universally accepted definition
- “Everything a vendor ships when you order an operating system” is good approximation
  - But varies wildly
- “The one program running at all times on the computer” is the **kernel**. Everything else is either a system program (ships with the operating system) or an application program.
- Hardware can execute instructions without an OS, but only an OS makes the system usable, safe, and efficient

# Operating-System Operations

- Interrupt driven by hardware
- Software error or request creates **exception** or **trap**
  - Division by zero, request for operating system service
- Other process problems include infinite loop, processes modifying each other or the operating system

## Operating-System Operations(2/2)

- **Dual-mode** operation allows OS to protect itself and other system components
  - **User mode** and **kernel mode**
  - **Mode bit** provided by hardware
    - Provides ability to distinguish when system is running user code or kernel code
    - Some instructions designated as **privileged**, only executable in kernel mode
    - System call changes mode to kernel, return from call resets it to user

# Operating-System Operations

- Bootstrap program – simple code to initialize the system, load the kernel
- Kernel loads
- Starts **system daemons** (services provided outside of the kernel)
- Kernel **interrupt driven** (hardware and software)
  - Hardware interrupt by one of the devices
  - Software interrupt (**exception** or **trap**):
    - Software error (e.g., division by zero)
    - Request for operating system service – **system call**
    - Other process problems include infinite loop, processes modifying each other or the operating system

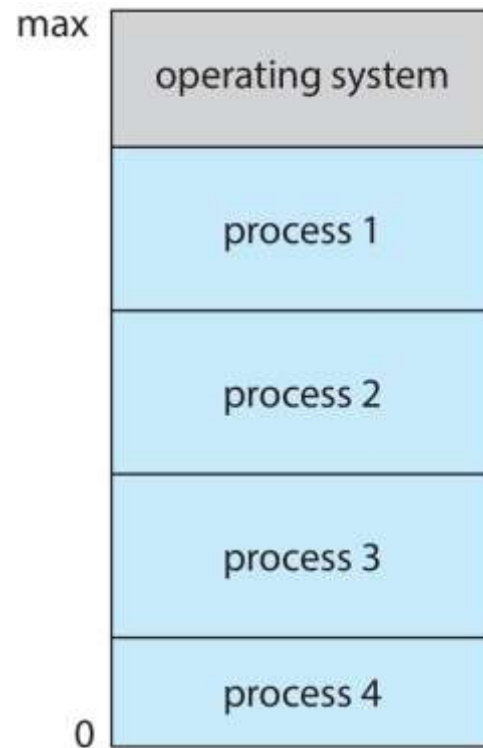
# Multiprogramming (Batch system)

- Single user cannot always keep CPU and I/O devices busy
- Multiprogramming organizes jobs (code and data) so CPU always has one to execute
- A subset of total jobs in system is kept in memory
- One job selected and run via **job scheduling**
- When job has to wait (for I/O for example), OS switches to another job

# Multitasking (Timesharing)

- A logical extension of Batch systems– the CPU switches jobs so frequently that users can interact with each job while it is running, creating **interactive** computing
  - **Response time** should be < 1 second
  - Each user has at least one program executing in memory  $\Rightarrow$  **process**
  - If several jobs ready to run at the same time  $\Rightarrow$  **CPU scheduling**
  - If processes don't fit in memory, **swapping** moves them in and out to run
  - **Virtual memory** allows execution of processes not completely in memory

# Memory Layout for Multiprogrammed System



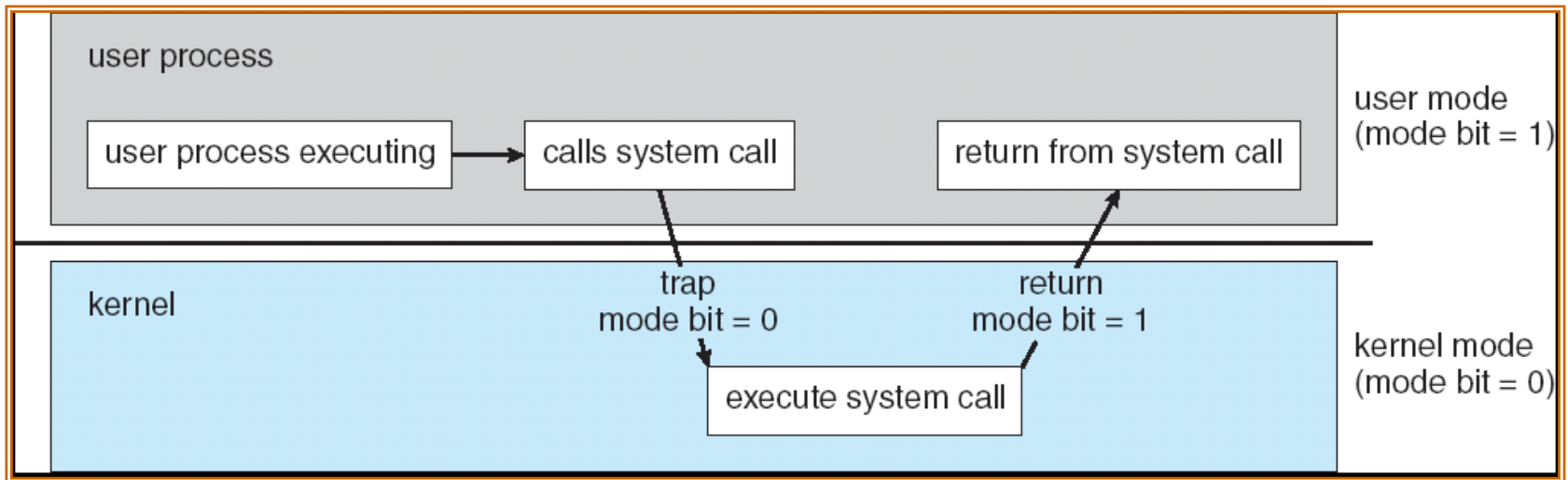
# Dual-mode Operation

- **Dual-mode** operation allows OS to protect itself and other system components
  - **User mode** and **kernel mode**
- **Mode bit** provided by hardware
  - Provides ability to distinguish when system is running user code or kernel code.
  - When a user is running  $\Rightarrow$  mode bit is “user”
  - When kernel code is executing  $\Rightarrow$  mode bit is “kernel”
- How do we guarantee that user does not explicitly set the mode bit to “kernel”?
  - System call changes mode to kernel, return from call resets it to user
- Some instructions designated as **privileged**, only executable in kernel mode

# Transition from User to Kernel Mode

- Timer to prevent infinite loop / process hogging resources
  - Set interrupt after specific period
  - Operating system decrements counter
  - When counter zero generate an interrupt
  - Set up before scheduling process to regain control or terminate program that exceeds allotted time

# Transition from User to Kernel Mode



# Timer

- Timer to prevent infinite loop (or process hogging resources)
  - Timer is set to interrupt the computer after some time period
  - Keep a counter that is decremented by the physical clock
  - Operating system set the counter (privileged instruction)
  - When counter zero generate an interrupt
  - Set up before scheduling process to regain control or terminate program that exceeds allotted time

# Operating System Structure

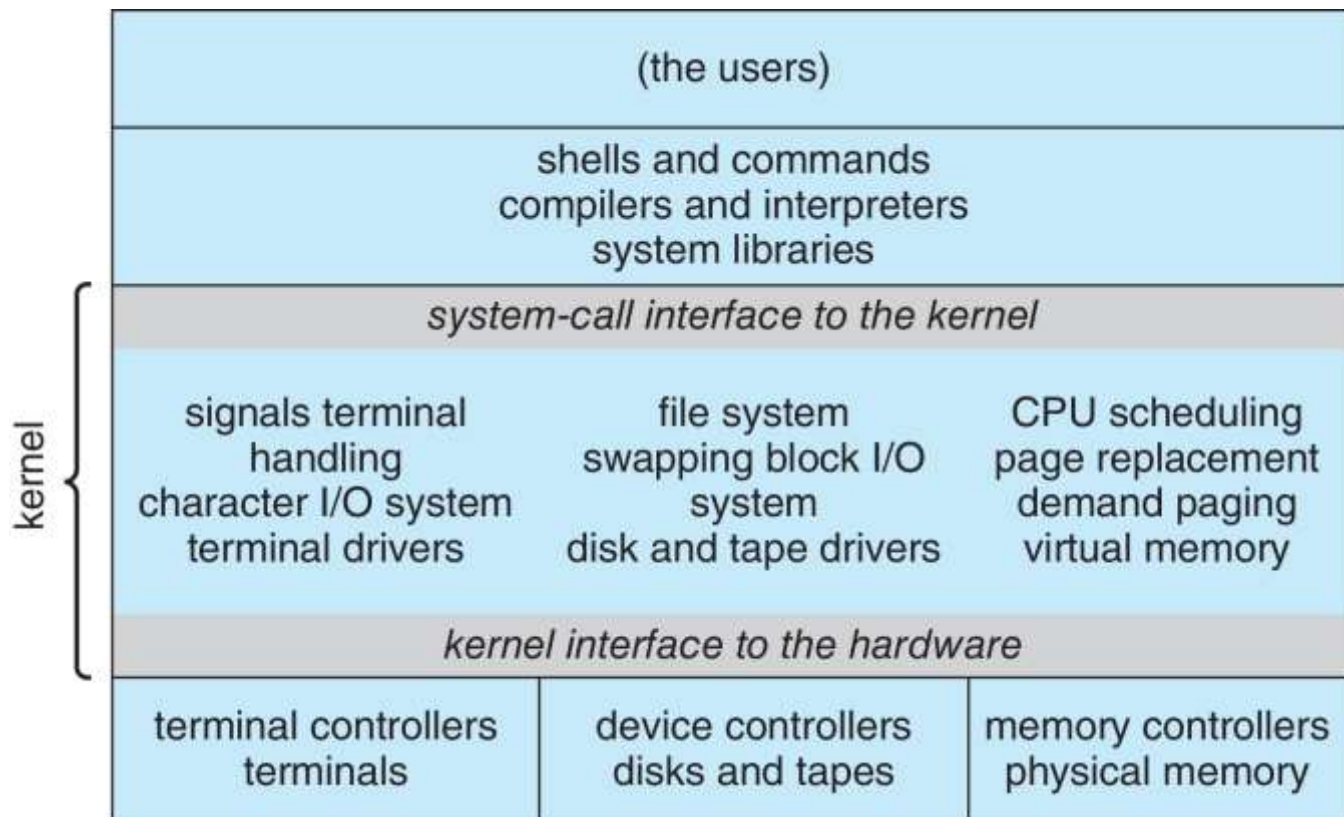
- General-purpose OS is very large program
- Various ways to structure ones
  - Simple structure – MS-DOS
  - More complex – UNIX
  - Layered – an abstraction
  - Microkernel – Mach

## Monolithic Structure – Original UNIX

- UNIX – limited by hardware functionality, the original UNIX operating system had limited structuring.
- The UNIX OS consists of two separable parts
  - Systems programs
  - The kernel
    - Consists of everything below the system-call interface and above the physical hardware
    - Provides the file system, CPU scheduling, memory management, and other operating-system functions; a large number of functions for one level

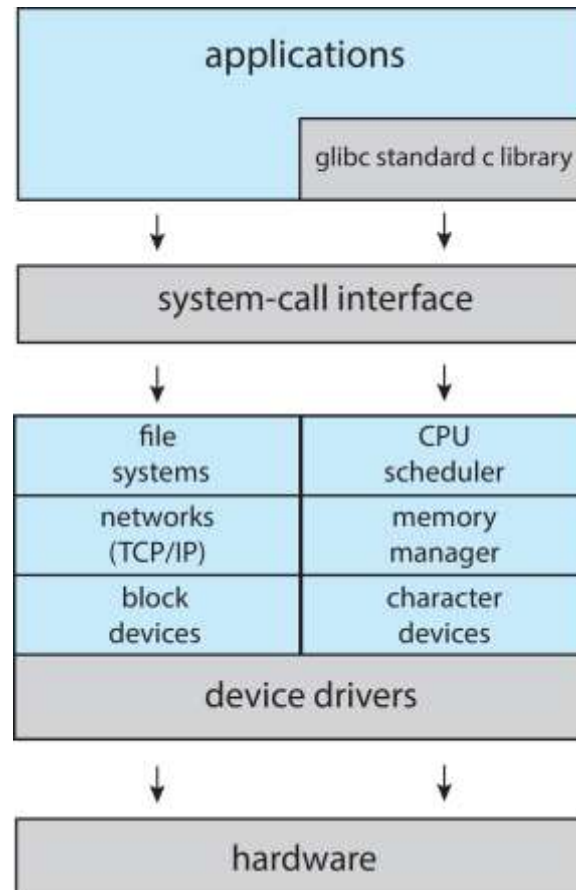
# Traditional UNIX System Structure

Beyond simple but not fully layered



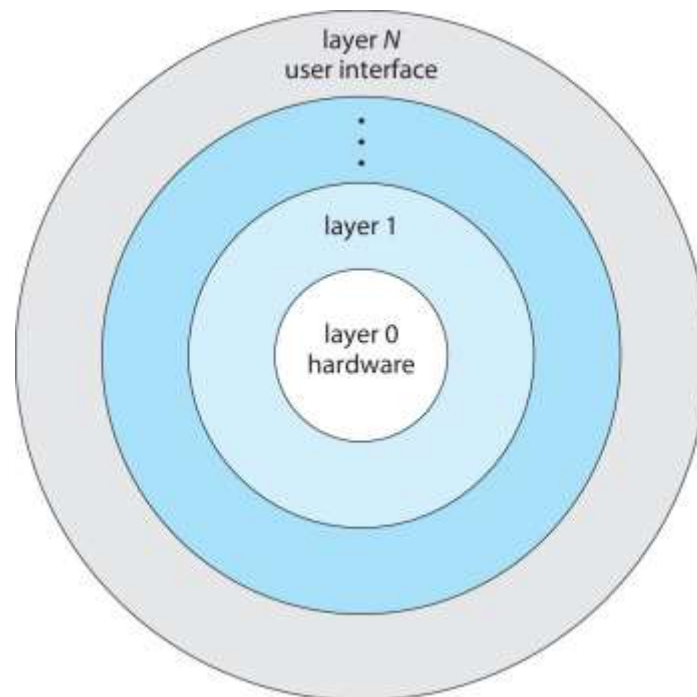
# Linux System Structure

Monolithic plus modular design



# Layered Approach

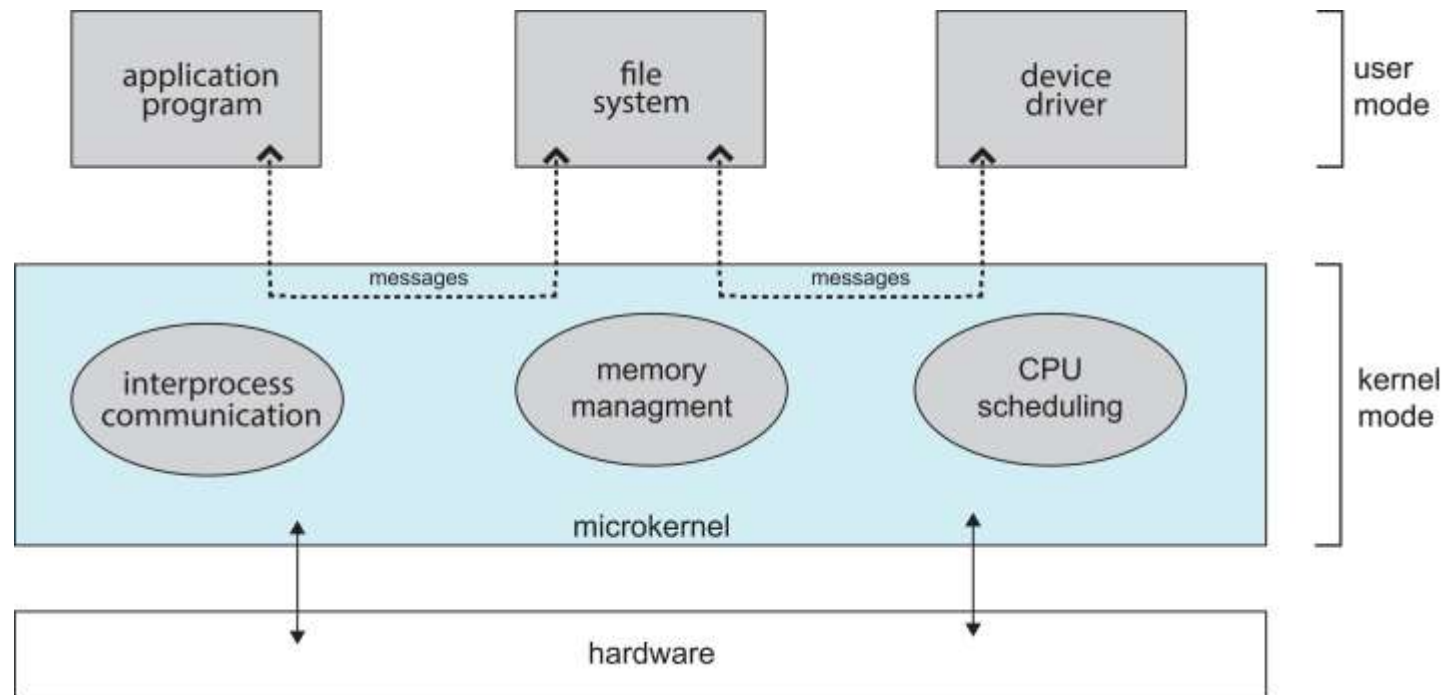
- The operating system is divided into a number of layers (levels), each built on top of lower layers. The bottom layer (layer 0), is the hardware; the highest (layer N) is the user interface.
- With modularity, layers are selected such that each uses functions (operations) and services of only lower-level layers



# Microkernels

- Moves as much from the kernel into user space
- **Mach** is an example of **microkernel**
  - Mac OS X kernel (**Darwin**) partly based on Mach
- Communication takes place between user modules using **message passing**
- Benefits:
  - Easier to extend a microkernel
  - Easier to port the operating system to new architectures
  - More reliable (less code is running in kernel mode)
  - More secure
- Detriments:
  - Performance overhead of user space to kernel space communication

# Microkernel System Structure



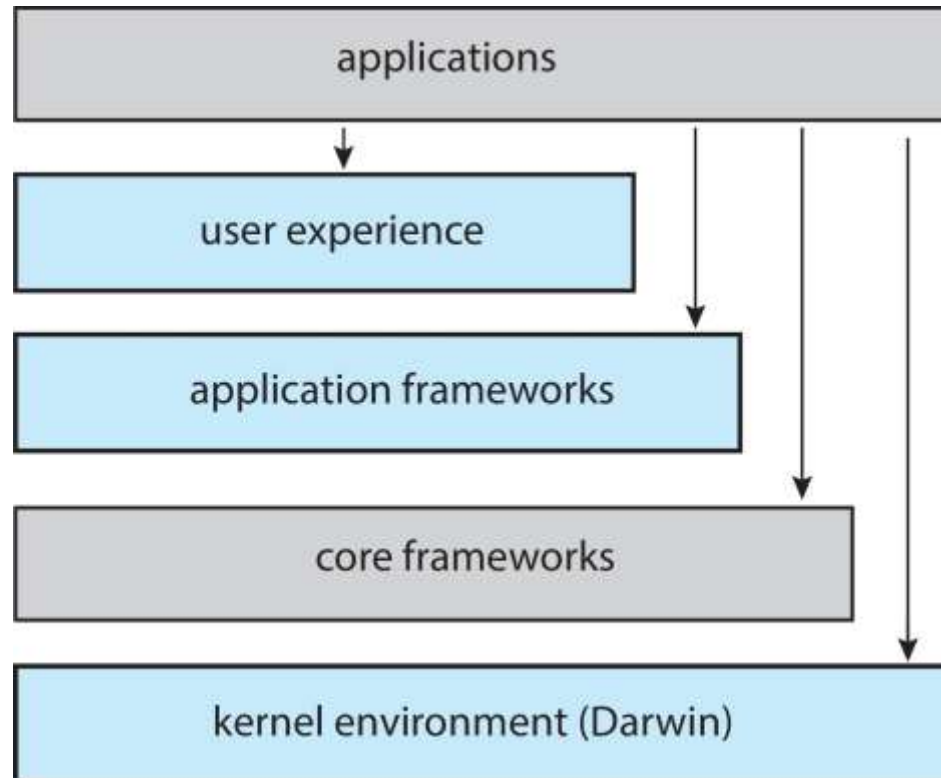
# Modules

- Many modern operating systems implement **loadable kernel modules (LKMs)**
  - Uses object-oriented approach
  - Each core component is separate
  - Each talks to the others over known interfaces
  - Each is loadable as needed within the kernel
- Overall, similar to layers but with more flexible
  - Linux, Solaris, etc.

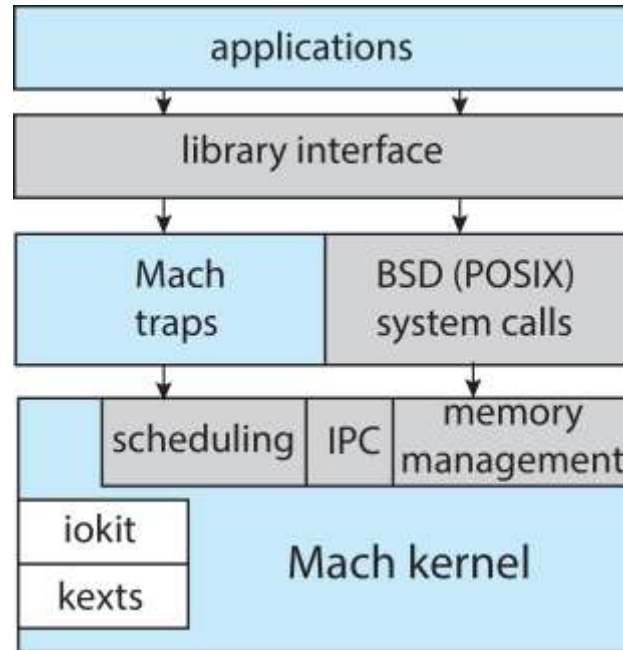
# Hybrid Systems

- Most modern operating systems are not one pure model
  - Hybrid combines multiple approaches to address performance, security, usability needs
  - Linux and Solaris kernels in kernel address space, so monolithic, plus modular for dynamic loading of functionality
  - Windows mostly monolithic, plus microkernel for different subsystem *personalities*
- Apple Mac OS X hybrid, layered, Aqua UI plus Cocoa programming environment
  - Below is kernel consisting of Mach microkernel and BSD Unix parts, plus I/O kit and dynamically loadable modules (called kernel extensions)

# macOS and iOS Structure



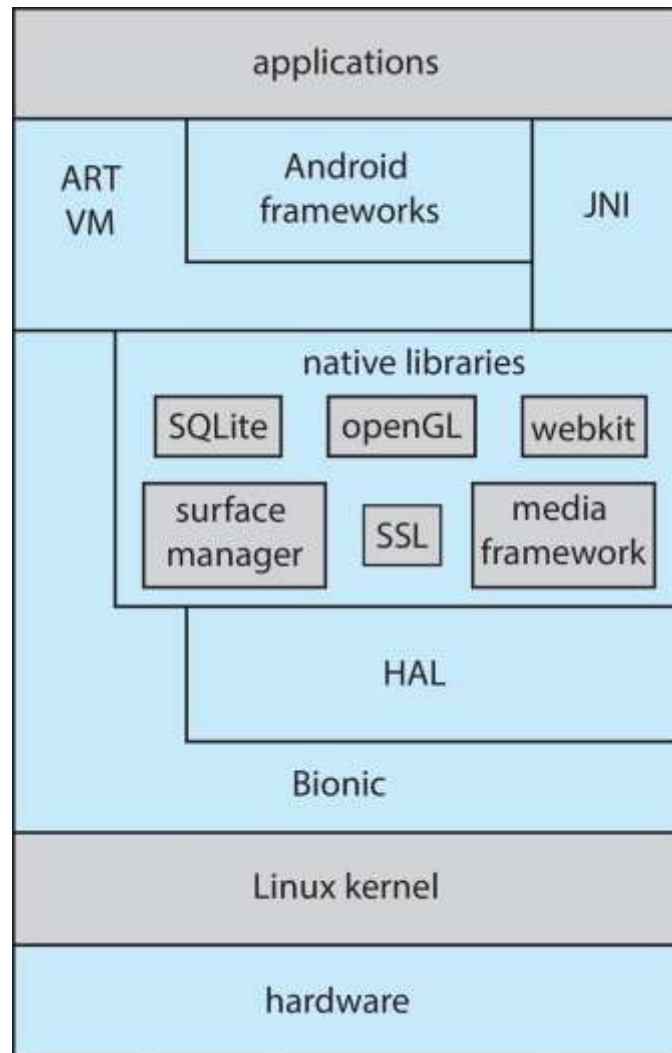
# Darwin



# Android

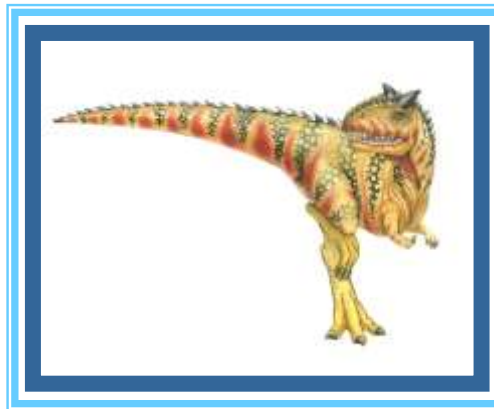
- Developed by Open Handset Alliance (mostly Google)
  - Open Source
- Similar stack to iOS
- Based on Linux kernel but modified
  - Provides process, memory, device-driver management
  - Adds power management
- Runtime environment includes core set of libraries and Dalvik virtual machine
  - Apps developed in Java plus Android API
    - Java class files compiled to Java bytecode then translated to executable thnn runs in Dalvik VM
- Libraries include frameworks for web browser (webkit), database (SQLite), multimedia, smaller libc

# Android Architecture



# Chapter 3: Processes

---



# Process Management Activities

The operating system is responsible for the following activities in connection with process management:

- Creating and deleting both user and system processes
- Suspending and resuming processes
- Providing mechanisms for process synchronization
- Providing mechanisms for process communication
- Providing mechanisms for deadlock handling

# Chapter 3: Processes

- 3.1 Process Concept
- 3.2 Process Scheduling
- 3.3 Operations on Processes
- 3.4 Inter-process Communication

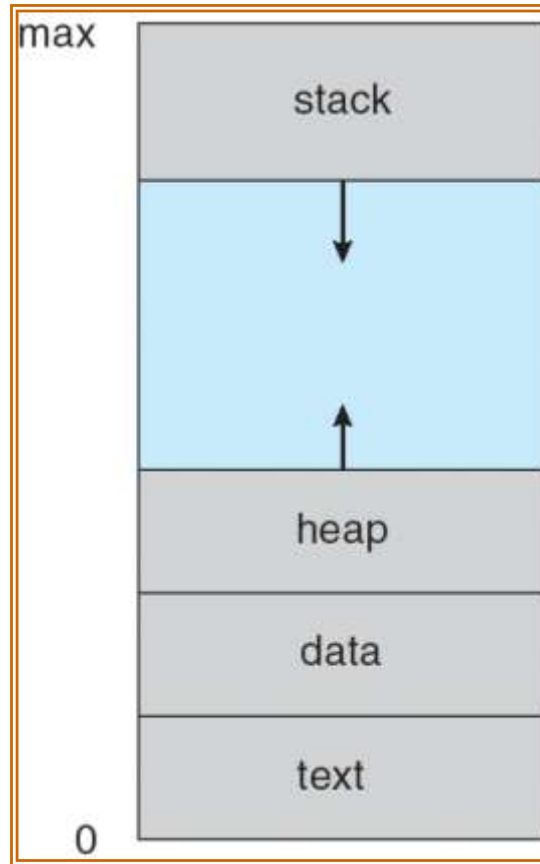
# Process Concept

- An operating system executes a variety of programs:
  - Batch system – jobs
  - Time-shared systems – user programs or tasks
- Textbook uses the terms *job* and *process* almost interchangeably

# Process Concept

- **Process – a program in execution**; process execution must progress in sequential fashion
- A process includes:
  - Text section
  - program counter
  - stack
  - data section
  - Heap

# Process in Memory



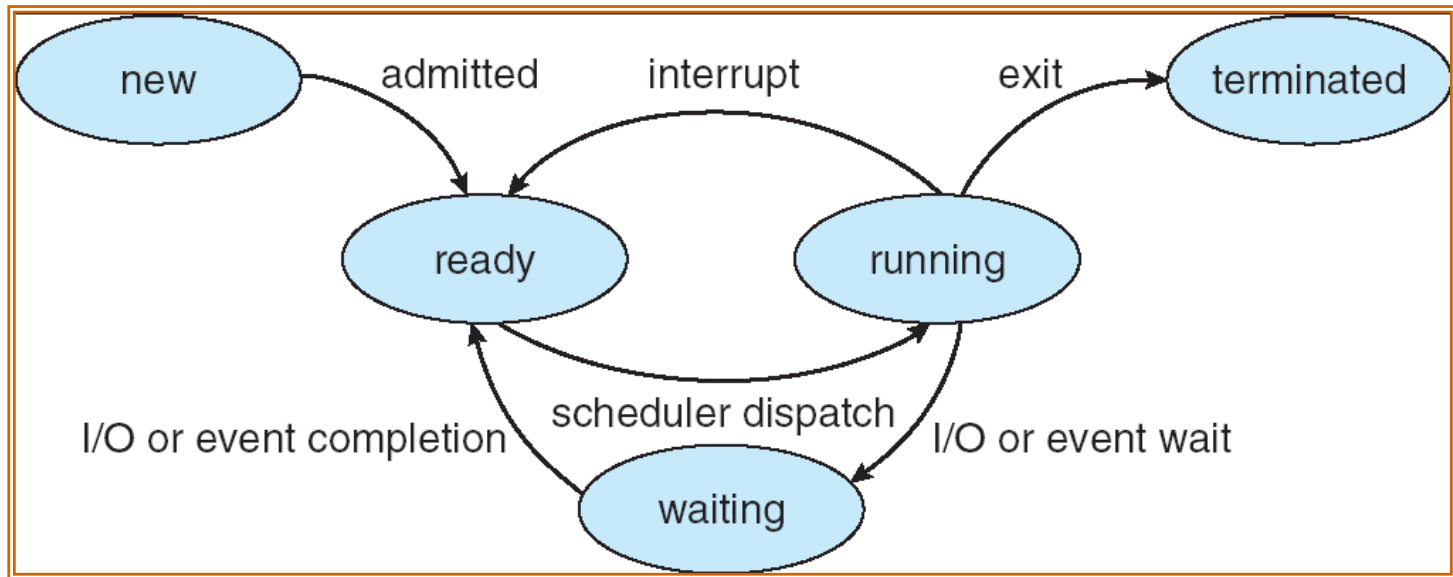
## Process in Memory

- Program is a **passive** entity
- Process is an active entity -- (program counter showing next instruction to execute and a set of associated resources)
- Program to process.
  - Double click on the executable file
  - Entering the name of the executable file on the command line

# Process State

- As a process executes, it changes *state*
  - **new**: The process is being created
  - **running**: Instructions are being executed
  - **waiting**: The process is waiting for some event to occur
  - **ready**: The process is waiting to be assigned to a processor
  - **terminated**: The process has finished execution

# Diagram of Process State



# Process Control Block (PCB)

PCB is also called as task control block.

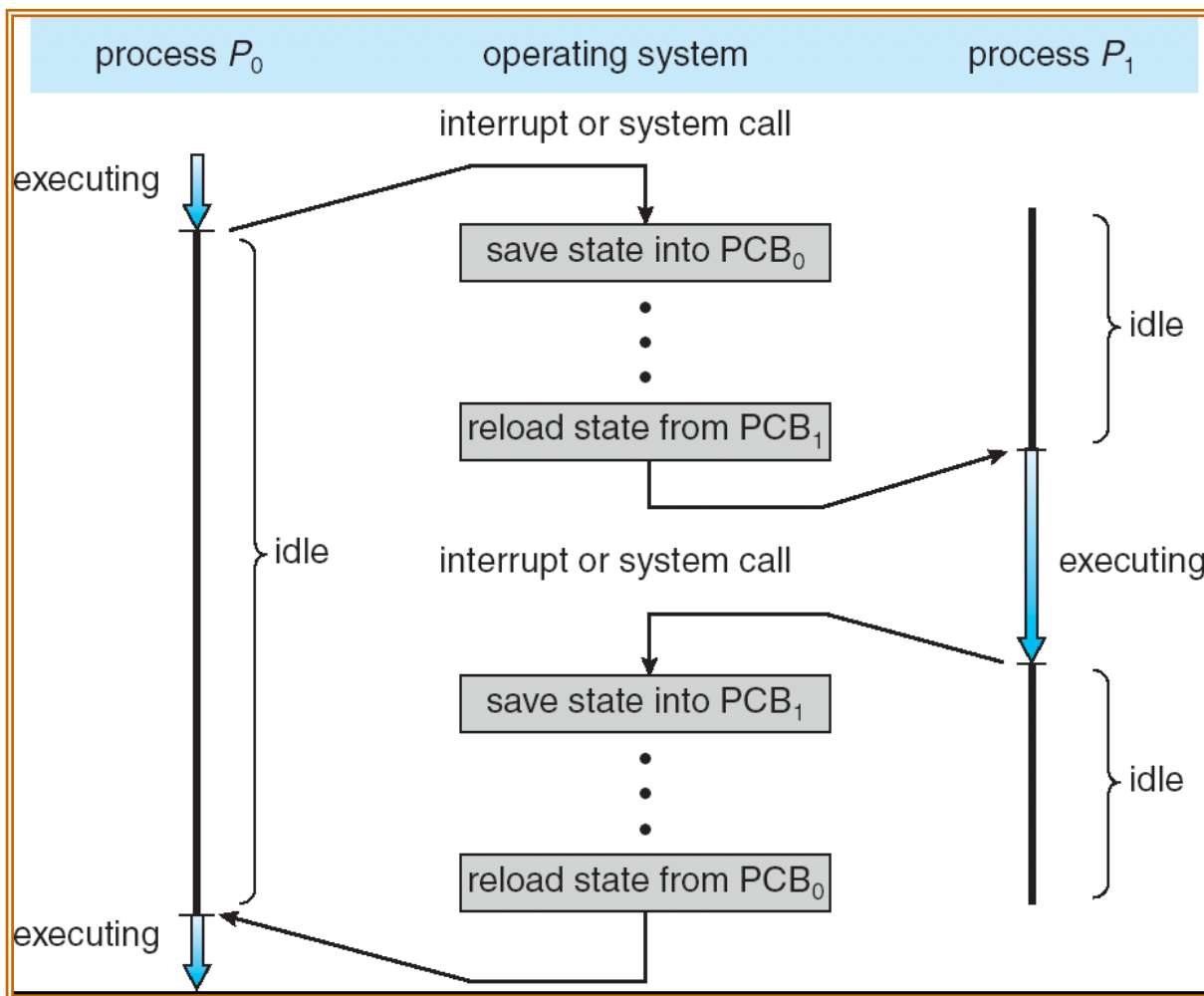
Information associated with each process

- Process state – new, ready, running, waiting, terminated
- Program counter – indicates the address of the next instruction to be executed
- CPU registers – accumulators, index register, stack pointers, general purpose registers
- CPU scheduling information – process priority, pointers to scheduling queues
- Memory-management information – base and limit registers, page tables
- Accounting information – amount of CPU and Real time used, account numbers
- I/O status information – list of I/O devices allocated to the process, list of open files.

# Process Control Block (PCB)



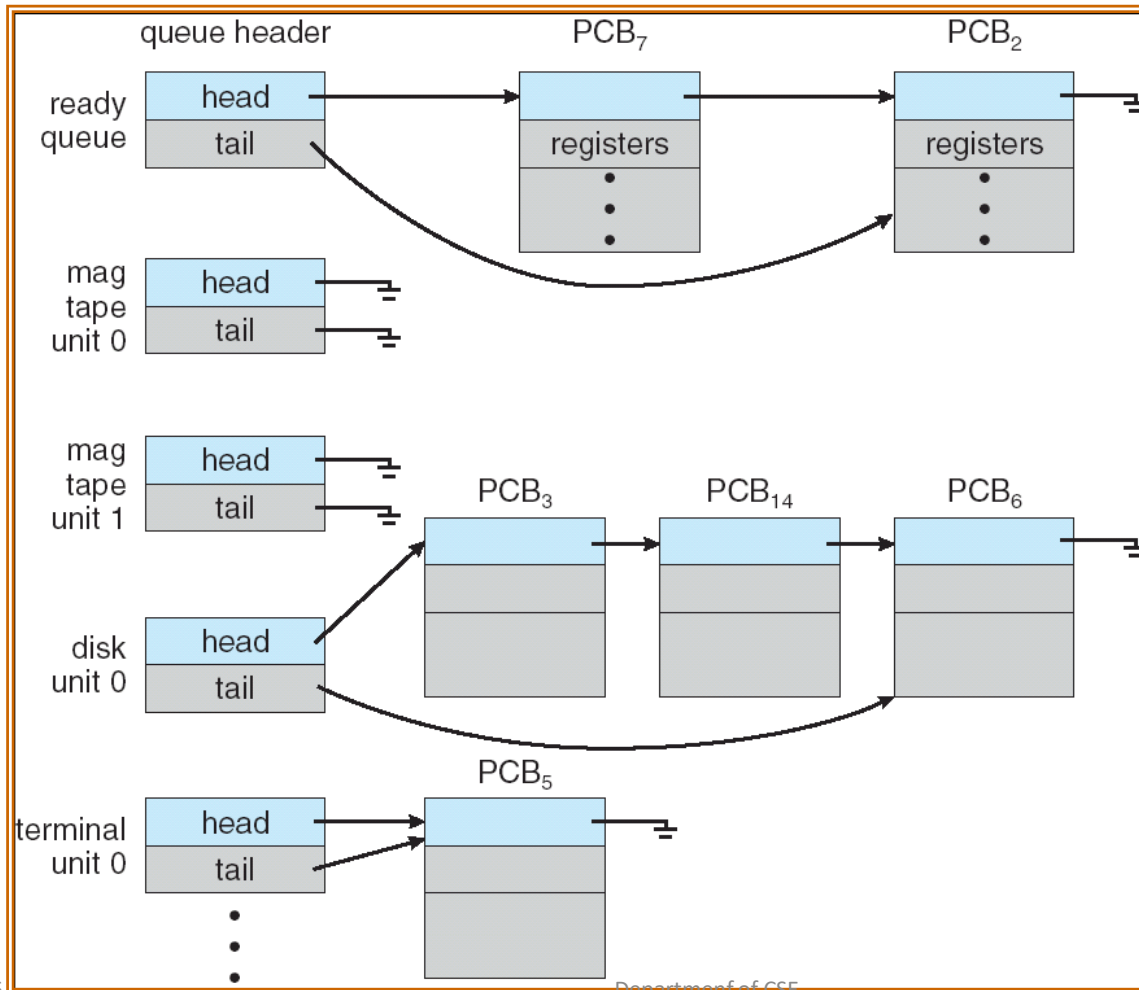
# CPU Switch From Process to Process



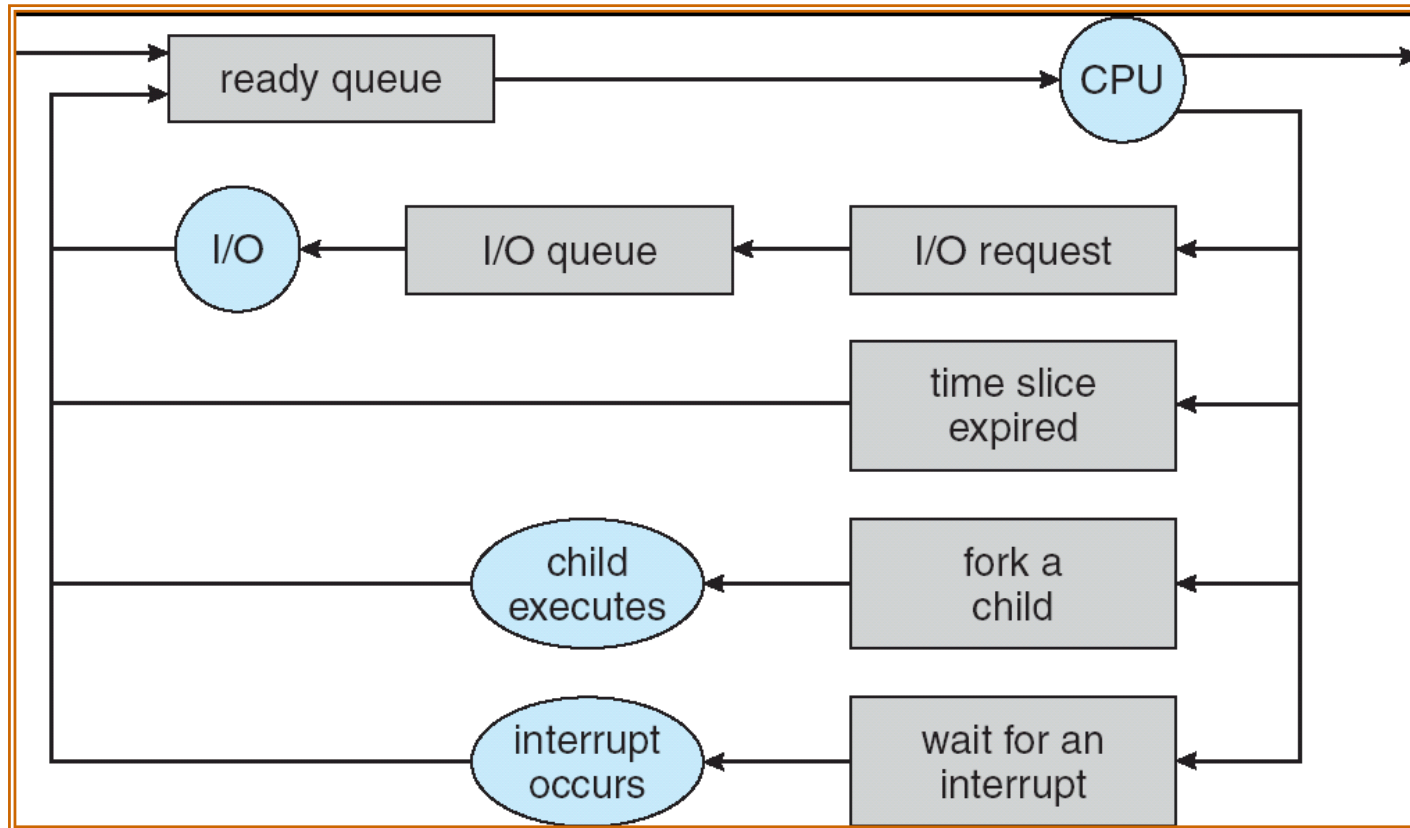
# Process Scheduling Queues

- **Job queue** – set of all processes in the system -100
- **Ready queue** – set of all processes residing in main memory -20, ready and waiting to execute
- **Device queues** – set of processes waiting for an I/O device
- Processes migrate among the various queues

# Ready Queue And Various I/O Device Queues



# Representation of Process Scheduling



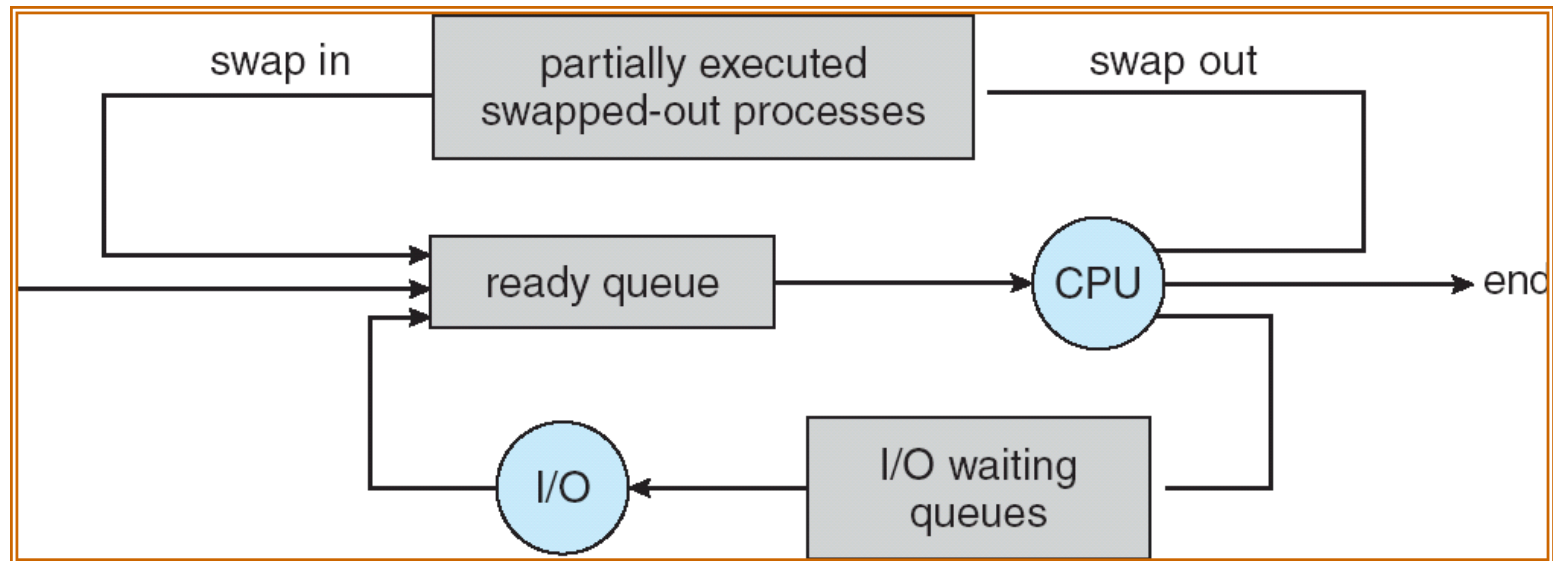
# Schedulers

- Process migrates from one queue to another, OS must select processes from these queues in some fashion.
- Selection process is carried out by **scheduler**.
  - **Long-term scheduler** (or job scheduler) – selects which processes should be brought into the ready queue
  - **Short-term scheduler** (or CPU scheduler) – selects which process should be executed next and allocates CPU.
    - Controls the degree of multiprogramming

# Schedulers

- Difference is,
  - **Frequency of execution** – short-term executes frequently long term much less frequently.
- Why?
  - Short term scheduler must select a new process for the CPU frequently.
  - Long term scheduler may need to be invoked only when a process leaves the system.
- CPU bound processes – COMPUTATION /PROCESSOR
- I/O bound processes – SPEND TIME PERFORMING I/O OPERATIONS

# Addition of Medium Term Scheduling



## Schedulers (Cont.)

- Short-term scheduler is invoked very frequently (milliseconds)  $\Rightarrow$  (must be fast)
- Long-term scheduler is invoked very infrequently (seconds, minutes)  $\Rightarrow$  (may be slow)
- The long-term scheduler controls the *degree of multiprogramming*
- Processes can be described as either:
  - **I/O-bound process** – spends more time doing I/O than computations, many short CPU bursts
  - **CPU-bound process** – spends more time doing computations; few very long CPU bursts

# Context Switch

- When CPU switches to another process, the system must save the state of the old process and load the saved state for the new process
- Context-switch time is overhead; the system does no useful work while switching
- Time dependent on hardware support

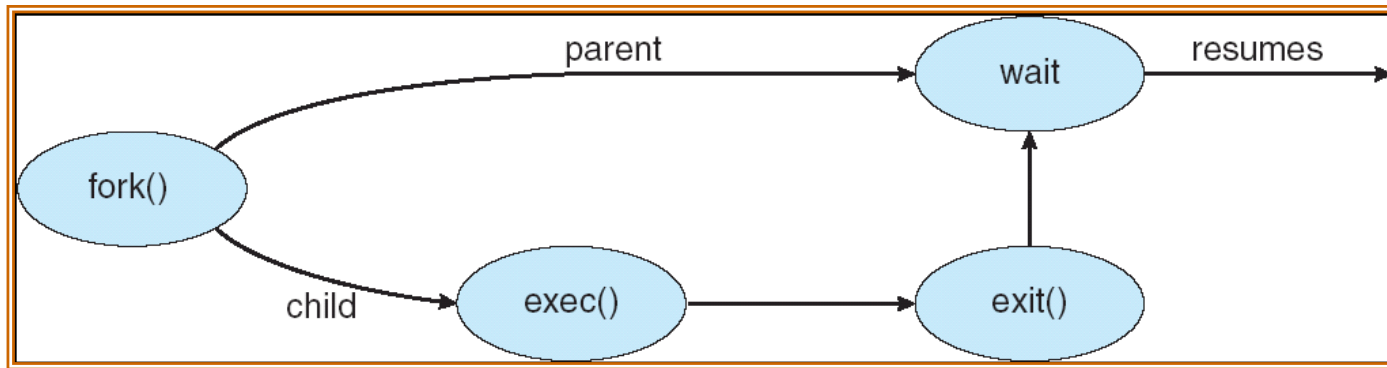
# Process Creation

- Parent process create children processes, which, in turn create other processes, forming a tree of processes
- **Resource sharing**
  - Parent and children share all resources
  - Children share subset of parent's resources
  - Parent and child share no resources
- **Execution**
  - Parent and children execute concurrently
  - Parent waits until children terminate

# Process Creation (Cont.)

- Address space
  - Child duplicate of parent
  - Child has a program loaded into it
- UNIX examples
  - **fork** system call creates new process
  - **exec** system call used after a **fork** to replace the process' memory space with a new program

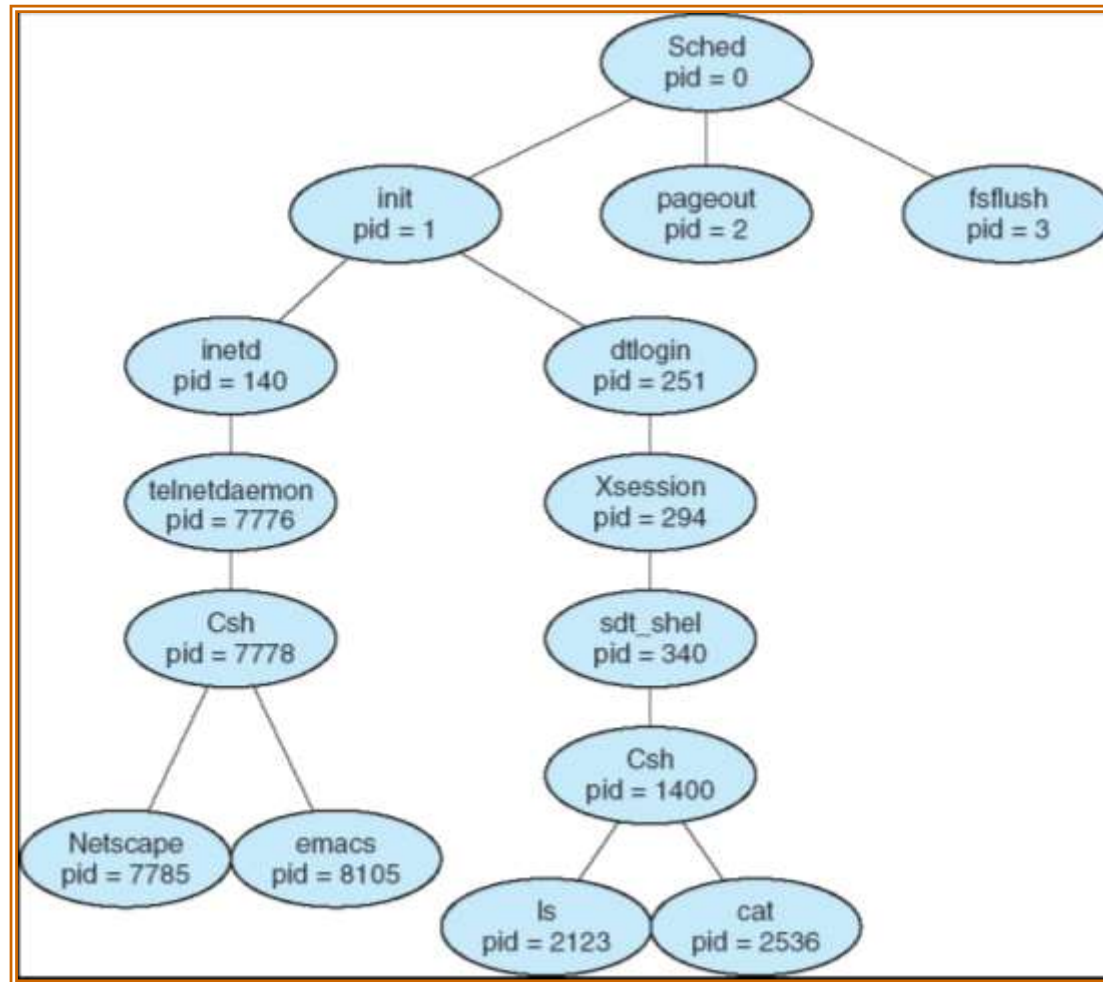
# Process Creation



# C Program Forking Separate Process

```
int main()
{
    pid_t pid;
    /* fork another process */
    pid = fork();
    if (pid < 0) { /* error occurred */
        fprintf(stderr, "Fork Failed");
        exit(-1);
    }
    else if (pid == 0) { /* child process */
        execlp("/bin/ls", "ls", NULL);
    }
    else { /* parent process */
        /* parent will wait for the child to complete */
        wait (NULL);
        printf ("Child Complete");
        exit(0);
    }
}
```

# A tree of processes on a typical Solaris



# Process Termination

- Process executes last statement and asks the operating system to delete it (**exit**)
  - Output data from child to parent (via **wait**)
  - Process' resources are deallocated by operating system
- Parent may terminate execution of children processes (**abort**)
  - Child has exceeded allocated resources
  - Task assigned to child is no longer required
  - If parent is exiting
    - Some operating system do not allow child to continue if its parent terminates
      - All children terminated - *cascading termination*

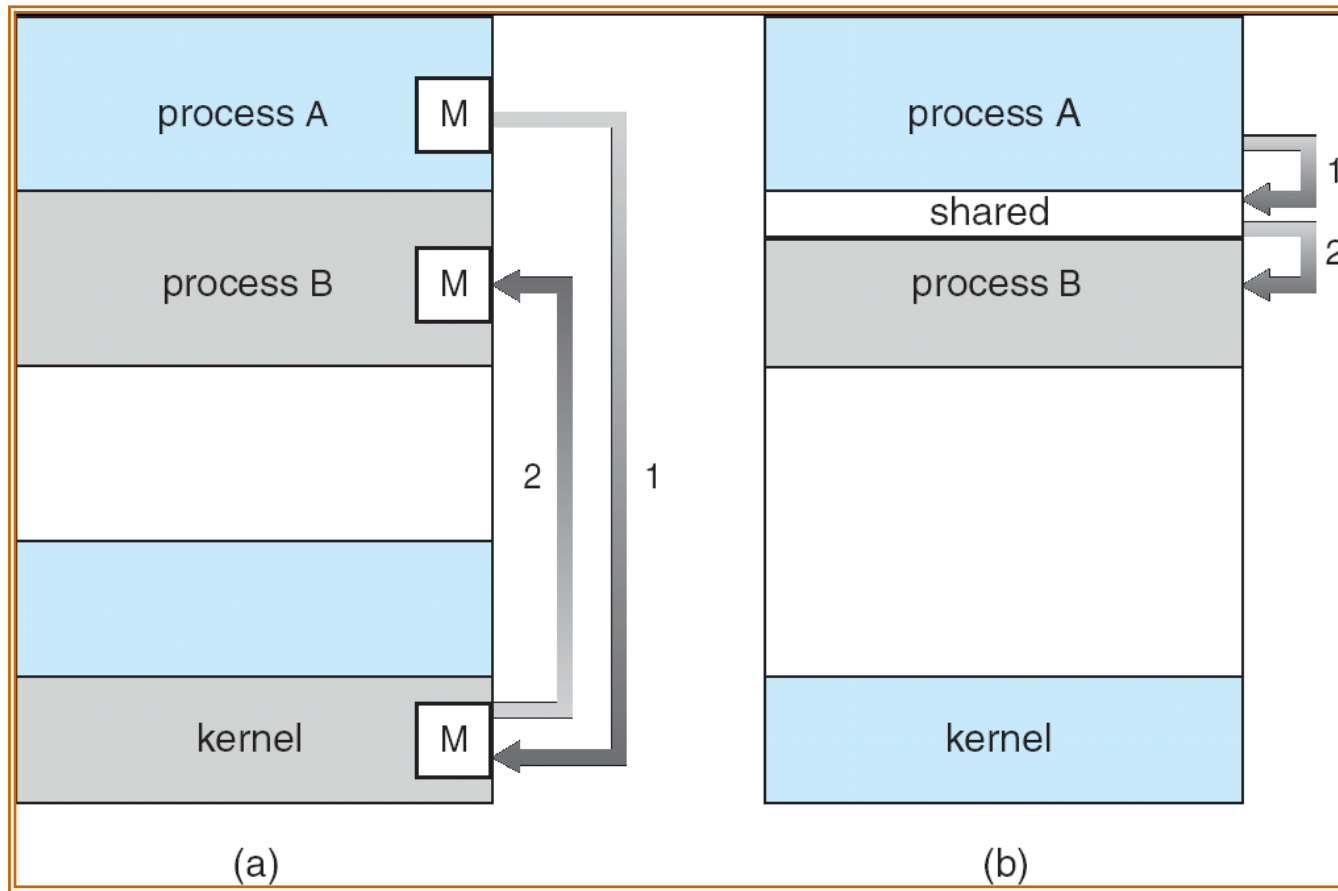
# Interprocess Communication (IPC)

- Mechanism for processes to communicate and to synchronize their actions
- Message system – processes communicate with each other without resorting to shared variables

# Interprocess Communication (IPC)

- IPC facility provides two operations:
  - **send**(*message*) – message size fixed or variable
  - **receive**(*message*)
- If  $P$  and  $Q$  wish to communicate, they need to:
  - establish a *communication link* between them
  - exchange messages via send/receive
- Implementation of communication link
  - physical (e.g., shared memory, hardware bus)
  - logical (e.g., logical properties)

# Communications Models



# Chapter 4: Threads



# Chapter 4: Threads

- 4.1 Overview
- 4.2 Multithreading Models

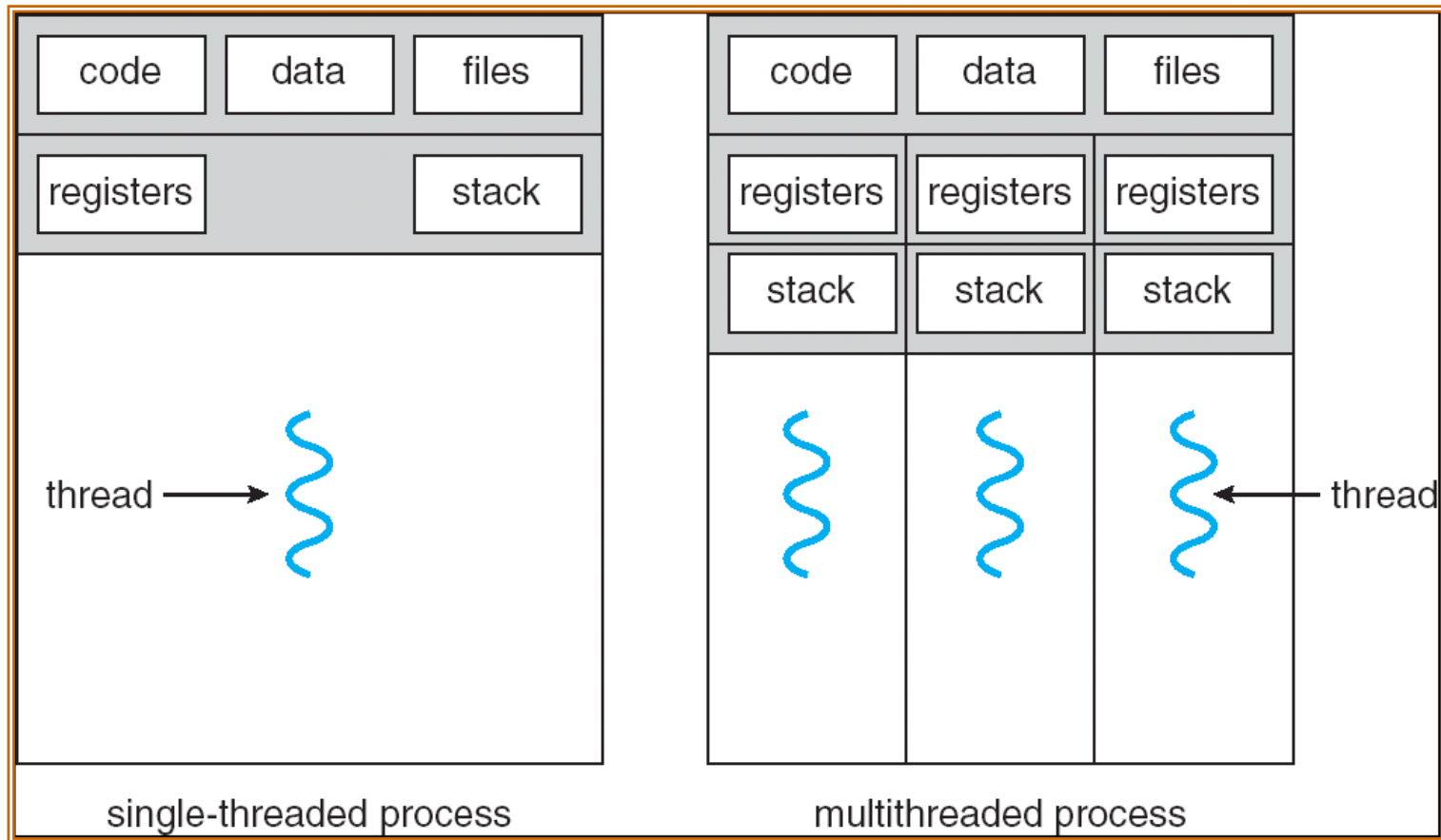
# Threads- Overview

- Thread is a basic unit of CPU utilization.
- Comprises of thread id, a program counter, a register set and a stack.
- Shares with other threads belonging to the same process its code section, data section and other OS resources.
- Traditional or heavyweight process has a single thread of control.
- With multiple threads, a process can perform more than one task at a time.

# Threads

- **Threads** differ from traditional multitasking operating system processes in that:
  - Processes are typically independent, while threads exist as subsets of a process
  - Processes carry considerably more state information than threads, whereas multiple threads within a process share process state as well as memory and other resources
  - Processes have separate address spaces, whereas threads share their address space
  - Processes interact only through system-provided inter-process communication mechanisms
  - Context switching between threads in the same process is typically faster than context switching between processes.

# Single and Multithreaded Processes



# Benefits

- Responsiveness
- Resource Sharing
- Economy
- Utilization of MP Architectures

# User Threads

- Thread management done by user-level threads library without kernel support.
- Three primary thread libraries:
  - POSIX Pthreads
  - Win32 threads
  - Java threads

# Kernel Threads

- Supported and managed directly by the Operating System.
- Examples
  - Windows XP/2000
  - Solaris
  - Linux
  - Tru64 UNIX
  - Mac OS X

# Multithreading Models

- Many-to-One
- One-to-One
- Many-to-Many

# Many-to-One

- Many user-level threads mapped to single kernel thread.

## **Advantage:**

- Efficient as thread management is done by the thread library in user space.

## **Drawback:**

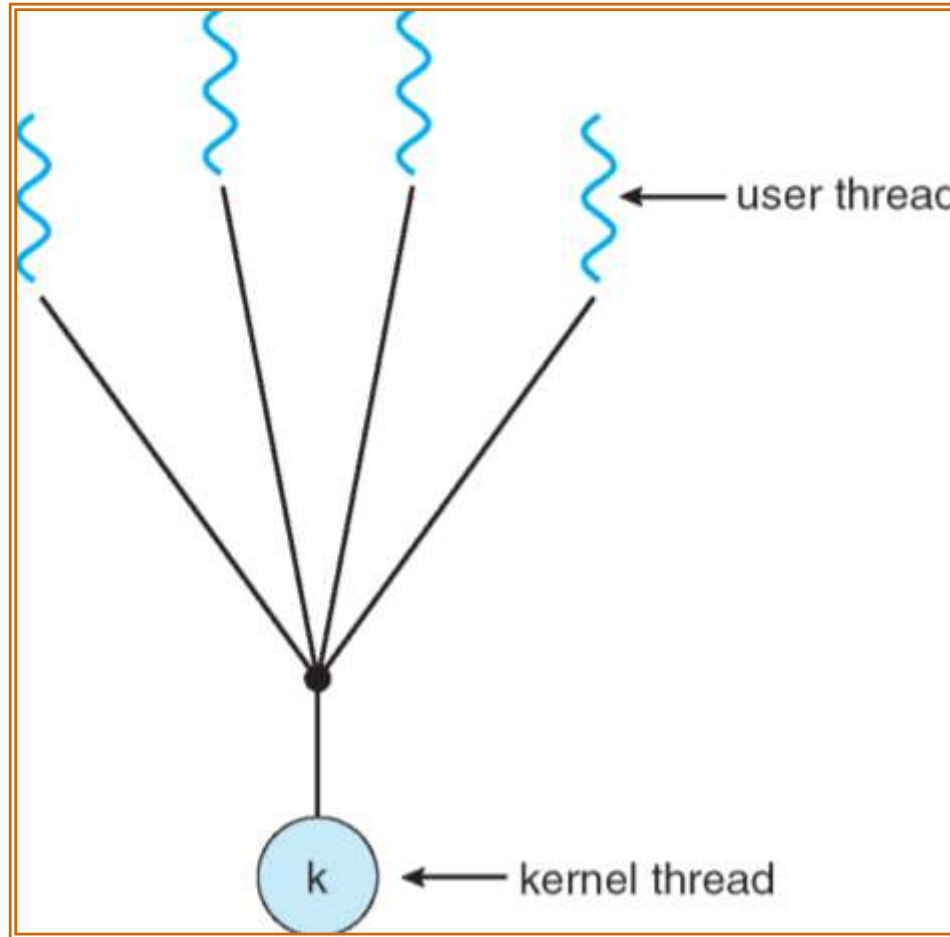
- If a thread makes a blocking system call, entire process will block.
- Multiple threads are unable to run in parallel on multiprocessors as only one thread can access the kernel at a time.

# Many-to-One

## **Examples:**

- Solaris Green Threads
- GNU Portable Threads

# Many-to-One Model



# One-to-One

- Each user-level thread maps to kernel thread.

## **Advantage:**

- Allows multiple threads to run in parallel on multiprocessors.
- Provides Concurrency by allowing another thread to run when a thread makes a blocking system call.

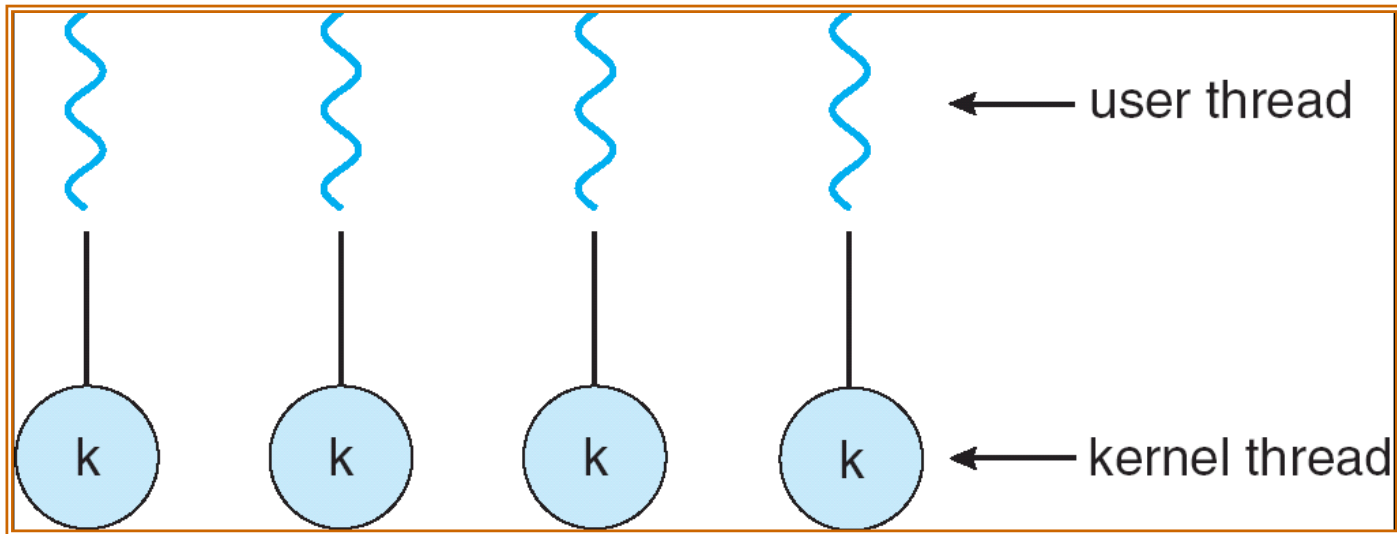
## **Drawback:**

- Overhead in the Creation of user thread for every kernel thread which burdens the performance of an application.

## **Examples**

- Windows NT/XP/2000
- Linux
- Solaris 9 and later

# One-to-one Model



# Many-to-Many Model

- Allows many user level threads to be mapped to many kernel threads

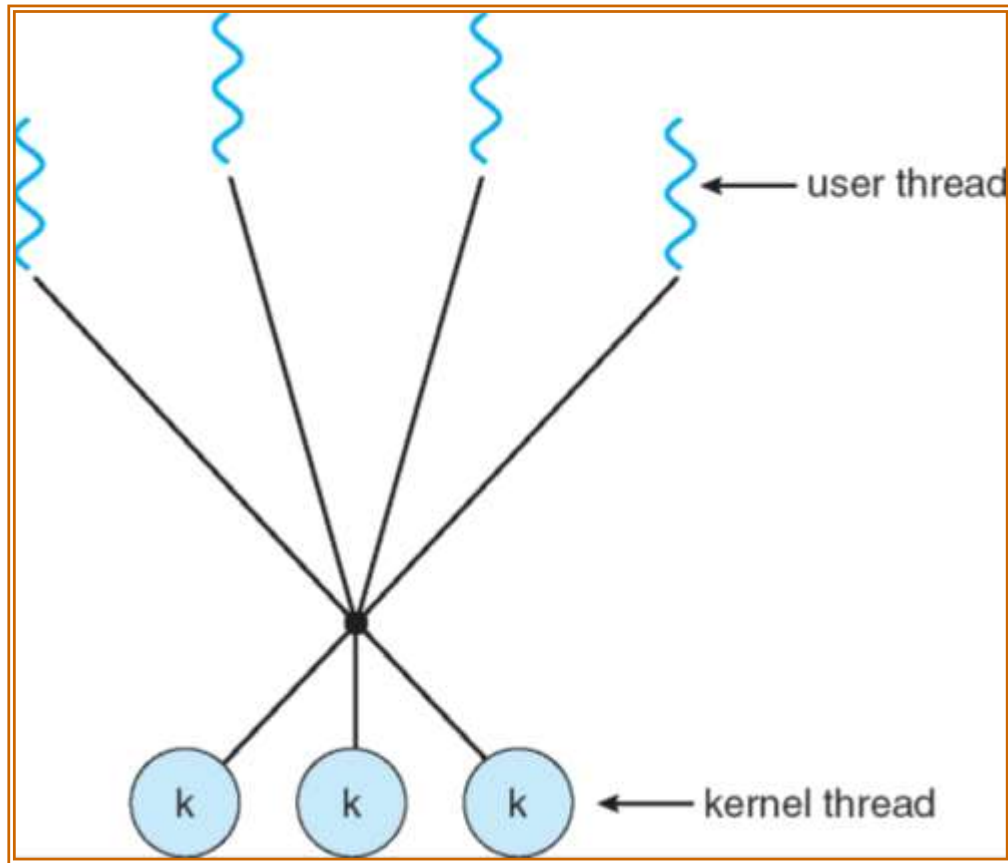
## **Advantage:**

- Allows the operating system to create a sufficient number of kernel threads
- Concurrency is achieved.
- When one thread performs a blocking system call, the kernel can schedule another thread.

## **Example:**

- Solaris prior to version 9
- Windows NT/2000 with the *ThreadFiber* package

# Many-to-Many Model



# Two-level Model

- Similar to M:M, except that it allows a user thread to be **bound** to kernel thread
- Examples
  - IRIX
  - HP-UX
  - Tru64 UNIX
  - Solaris 8 and earlier

# Two-level Model

